When Apps Outlive Vendors: Security Implications of IoT Abandonware

Dayeon Kang
dayeonkang@umass.edu
University of Massachusetts Amherst
Amherst, MA, USA

Elvis Yeboah-Duako
University of Massachusetts Amherst
Amherst, MA, USA
eyeboahduako@umass.edu

Sachin Thomas
University of Massachusetts Amherst
Amherst, MA, USA
sachinthomas@umass.edu

Pubali Datta
University of Massachusetts Amherst
Amherst, MA, USA
pdatta@umass.edu

A Artifact Appendix

A.1 Abstract

This artifact accompanies the paper "When Apps Outlive Vendors: Security Implications of IoT Abandonware." It contains the dataset and the analysis framework used to study 61,500 abandoned Android IoT companion apps, i.e., apps that were removed from Google Play or not updated for over two years as of March 2025. The artifact is a public Git repository, archived on Zenodo, that bundles (i) app lists in a CSV file, (ii) Python scripts that construct the dataset, (iii) pipelines for outdated-dependency/CVE analysis, at-risk-domain analysis (reachability, blocklists, ownership change), and permission/sensor analysis, and (iv) scripts and notebooks that post-process IoTFlow data-flow and cryptographic analysis. The scripts, excuting on decompiled APKs, generate JSON/JSONL/CSV results and the figures reported in the paper. APKs are not redistributed and must be fetched from AndroZoo. The artifact is released for research use under the Creative Commons Attribution 4.0 International license (CC BY 4.0).

A.2 Description & Requirements

The `README.md` at the repository root provides script execution commands with the configurable path variables of each script listed in tables. The layout is as follows.

- `dataset/`: the app lists three CSV app lists (abandoned apps, abandoned apps with AndroZoo SHA256 hashes and versions, and the top-500 active-app baseline), plus four numbered scripts that construct the dataset (description retrieval, BERT IoT classification, IoT filtering, abandonment check).
- `src/cve-search/`: NVD database build, bundled-library extraction, offline CVE search, shard merging, CVE counting, and a plotting notebook.
- `src/permissions-urls-and-sensor-analysis/`: URL and sensor extraction, unique-domain extraction, DNS reachability, permission extraction and severity classification, and sensor counting/categorization.
- `src/blocklist-analysis/` and `src/domain-change-analysis/`: blocklist matching and WHOIS-based domain-ownership-change detection. Each has its own `README.md` and bundled reference data.
- `src/iotflow/`: IoTFlow data-flow and cryptographic analysis scripts, and plotting notebooks.

A.2.1 Security, privacy, and ethical concerns. All analyses are static: no app is installed or executed, and no device, server, or user is exploited. However, the APKs downloaded from AndroZoo are abandoned apps that may contain malware or adware, so they must never be installed on a personal device; decompile them with Jadx in an isolated VM or container. The reachability step sends DNS A-record queries for domains extracted from the apps to Google's public resolver (8.8.8.8) but does not contact those domains. The AndroZoo and WhoisXMLAPI require personal keys; set them locally and do not commit them. The IoTFlow step runs third-party Docker containers. No personal user data is collected by our scripts.

A.2.2 How to access. The persistent archive is on Zenodo (DOI https://doi.org/10.5281/zenodo.22840852, CC BY 4.0). A maintained copy is on BitBucket: https://bitbucket.org/umass-lab/iot-abandonware-analysis/. Some large files are stored with Git LFS, so run `git lfs install && git lfs pull` after cloning.

A.2.3 Hardware dependencies. Running the full pipeline on all 61,500 apps requires about 5 TB of disk space (about 1 TB of APKs and about 4 TB of decompiled sources), and a multi-core machine.

A.2.4 Software dependencies. Linux or macOS with Python 3 and the pip packages—`requests`, `google-play-scraper`, `tldextract`, `dnspython`, `matplotlib`, `ijson`, `nvdlib`, `packaging`, and `tqdm`; Jupyter for the notebooks; Git LFS; Jadx for decompilation; and Docker for IoTFlow.

A.2.5 Benchmarks. The datasets are the three CSV files in `dataset/` (61,500 abandoned apps and a 500-app active baseline, from an AndroZoo snapshot of March 2025. The abandoned-app CSVs list 61,529 package IDs, and 29 apps whose app category could not be found were omitted from the analyzed 61,500), the blocklist snapshots, and the cached WHOIS history.

A.3 Set Up

A.3.1 Installation. Clone the repository, fetch the LFS files, and install the Python packages: `git clone` the BitBucket repository above and run `git lfs pull` or download the archived repository on Zenodo. Then, `pip install requests google-play-scraper tldextract dnspython matplotlib ijson nvdlib packaging tqdm`. For the full pipeline, additionally download the APKs listed

in the version CSV of `dataset/` from AndroZoo, decompile them with `jadx -d <output_dir>/ <apk_file>.apk`, and edit the path variables at the top of each script. For IoTFlow, clone https://github.com/SecPriv/iotflow into `src/iotflow/` and follow the README.

A.3.2 Basic test. Run `src/domain-change-analysis/ownership_change_analysis.py`. It uses the cached WHOIS history, needs no API key, and processes 737 domains. It writes JSON and CSV outputs. A second test covers the blocklist scanner. In `src/blocklist-analysis/`, create a small JSON file that maps a package name to a few URLs (format in the README) and run `python url_analysis.py –input-glob sample.json –output-dir test_results`, answering `y` at the prompt. The scanner loads the nine blocklist categories from `blocklists/` and writes `results.csv` and `summary.csv`, in which URLs on a blocklist (e.g., known ad domains) are flagged. A third test runs `python check_oisd_blocklist.py` and `python url-haus-malicious-url-detector.py` inside `malicious-checks/` after `git lfs pull`. They use only local data, finish in seconds, and write `results/blocked_domains.json` and `results/apps_with_malicious_urls.json`.

A.4 Evaluation Workflow

Since this artifact is submitted for the Artifacts Available badge, we only outline the claims and pipelines. Full re-execution requires external resources (APKs, NVD feeds, AndroZoo access).

A.4.1 Major claims.

- **(C1)**: Most abandoned apps bundle outdated libraries with CVEs published after the app's last update (73.6% of the dataset). This is shown by (E1) in Section A.4.2.
- **(C2)**: Abandoned apps embed unreachable, blocklisted, or ownership-changed domains that are at risk of takeover. This is shown by (E2) in Section A.4.2.
- **(C3)**: Sensitive data flows to broken or hijackable endpoints, e.g., in 30 of the top 1,000 most-installed apps. This is shown by (E3) in Section A.4.2.

A.4.2 Experiments.

- **(E1)**: [CVE analysis; several compute-days on the full dataset] *Preparation:* decompile the APKs and download the NVD feeds. *Execution:* run the scripts in `src/cve-search/` in order (database build, library parsing, offline search, merge, count). *Results:* `cve_analysis_count.jsonl` and the notebook `plot-cve-analysis.ipynb`.
- **(E2)**: [Domain analysis; several compute-days] *Preparation:* decompiled APK files. *Execution:* run the scripts `1.1`–`1.4` in `src/permissions-urls-and-sensor-analysis/`, then `src/blocklist-analysis/url_analysis.py` and `src/domain-change-analysis/ownership_change_analysis.py`. *Results:* per-app reachability summaries, blocklist matches, and ownership-change outputs, which match the bundled reference results for the cached subset.
- **(E3)**: [IoTFlow analysis; several compute-days (depends on the number of virtual machines)] *Preparation:* clone IoTFlow into `src/iotflow/` and add APKs to `apps_to_analyze/`. *Execution:* run `docker compose up` in `FlowAnalysis/docker` and `VSA/docker`, then `flowanalysis.py` and `vsa_analysis.py`. *Results:* plot with `plot-flowanalysis.ipynb` and `plot-vsa-analysis.ipynb`.

A.5 Notes on Reusability

The pipelines are modular and each stage reads and writes plain JSON/JSONL/CSV files, so they can be applied to other app collections by changing the input lists and path variables. The permission, sensor, and blocklist scripts can be reused on any set of decompiled Android apps, and the domain-ownership-change detector relies only on registrar, nameserver, and DNSSEC signals that remain available under GDPR redaction. The CVE pipeline can be applied to any collection of decompiled apps by supplying a new app list, and rerunning it against a newer set of NVD feeds updates the matched CVEs. The local SQLite index is built once and searched in parallel shards. The IoTFlow post-processing and plotting scripts accept the output of IoTFlow for any APK set placed in `apps_to_analyze/`.

A.6 Version

Based on the LaTeX template for Artifact Evaluation V20220926.